DATABASE MANAGEMENT SYSTEM (DBMS)

TITLE PAGE

Subject: Database Management System (DBMS)
Level: Undergraduate (Beginner to Intermediate)
Description: This dataset contains structured and detailed DBMS concepts with explanations, examples, and definitions designed for Retrieval-Augmented Generation (RAG) systems.

TABLE OF CONTENTS

1. Introduction to DBMS ..... Page 1
2. Database Concepts and Types ..... Page 2
3. Data Models and ER Model ..... Page 3
4. Relational Model and Keys ..... Page 4
5. Normalization ..... Page 5
6. SQL Basics ..... Page 6

PAGE 1: INTRODUCTION TO DBMS

Definition:
A Database Management System (DBMS) is software that allows users to store, manage, and retrieve data efficiently. It acts as an interface between the user and the database, ensuring that data is consistently organized and accessible.

Need for DBMS:
Before DBMS, data was stored in file systems, which had several issues:

- Data redundancy (same data stored multiple times)
- Data inconsistency (different versions of same data)
- Difficulty in accessing data
- Lack of security

DBMS solves these problems by providing structured storage and controlled access.

Key Functions of DBMS:

- Data storage and organization
- Data retrieval using queries
- Data security and access control
- Backup and recovery
- Transaction management

Advantages:

- Reduces redundancy by storing data in one place
- Maintains consistency across data
- Supports multiple users simultaneously
- Improves data integrity

Example:
In a college system, student details, course details, and results are stored in a DBMS instead of separate files, allowing easy access and updates.

PAGE 2: DATABASE CONCEPTS AND TYPES

Database:
A database is a collection of logically related data stored in an organized manner. It is designed to manage large amounts of information efficiently.

Basic Concepts:

- Data: Raw facts (e.g., numbers, text)
- Information: Processed data (e.g., student report)
- Table: Collection of related data in rows and columns

Types of Databases:

1. **Hierarchical Database:**
   Data is organized in a tree-like structure where each record has one parent and multiple children.
   Example: Organization structure
2. **Network Database:**
   Data is connected through multiple relationships, allowing more complex connections than hierarchical.
   Example: Graph-based relationships
3. **Relational Database:**
   Data is stored in tables (relations). It is the most widely used type.
   Example: MySQL, PostgreSQL
4. **NoSQL Database:**
   Used for unstructured or semi-structured data such as JSON, documents, or key-value pairs.
   Example: MongoDB

Comparison:
Relational databases use structured tables, while NoSQL databases handle flexible and large-scale data.

PAGE 3: DATA MODELS AND ER MODEL

Data Model:
A data model defines how data is stored, organized, and accessed in a database. It provides a conceptual framework for database design.

Types of Data Models:

- Hierarchical Model
- Network Model
- Relational Model

ER Model (Entity-Relationship Model):
The ER model is used to visually represent the structure of a database.

Components of ER Model:

1. **Entity:**
   An object or real-world thing (e.g., Student, Course)
2. **Attribute:**
   A property of an entity (e.g., Student Name, Age)
3. **Relationship:**
   Association between entities (e.g., Student enrolls in Course)

Types of Relationships:

- One-to-One
- One-to-Many
- Many-to-Many

Example:
A Student entity with attributes (ID, Name) is related to Course through an "enrolls" relationship.

Importance:
The ER model helps in designing a clear and efficient database structure before implementation.

PAGE 4: RELATIONAL MODEL AND KEYS

Relational Model:
The relational model stores data in tables called relations. Each table consists of rows (tuples) and columns (attributes).

Structure of Table:

- Rows: Individual records
- Columns: Attributes or fields

Keys in DBMS:
Keys are used to identify and establish relationships between records.

Types of Keys:

1. **Primary Key:**

- Uniquely identifies each record
- Cannot be null

Example:
Student_ID

2. **Foreign Key:**

- Links one table to another
- Maintains referential integrity

Example:
Course_ID in Student table

3. **Candidate Key:**

- A set of attributes that can uniquely identify a record

4. **Composite Key:**

- Combination of two or more attributes

Example:
(Student_ID, Course_ID)

Importance of Keys:

- Prevent duplicate data
- Maintain relationships between tables
- Ensure data accuracy

PAGE 5: NORMALIZATION

Definition:
Normalization is the process of organizing data in a database to eliminate redundancy and improve data integrity.

Goals:

- Remove duplicate data
- Ensure data dependencies are logical
- Improve consistency

Normal Forms:

1. **First Normal Form (1NF):**

- Each column contains atomic (single) values
- No repeating groups

2. **Second Normal Form (2NF):**

- Must be in 1NF
- No partial dependency (all attributes depend on full primary key)

3. **Third Normal Form (3NF):**

- Must be in 2NF
- No transitive dependency (non-key attributes depend only on primary key)

Example:
Instead of storing student and course in one table, split into:

- Student table
- Course table
- Enrollment table

Advantages:

- Reduces redundancy
- Improves data consistency
- Makes database scalable

PAGE 6: SQL BASICS

Definition:
SQL (Structured Query Language) is used to interact with relational databases. It allows users to create, update, and retrieve data.

Types of SQL Commands:

1. DDL (Data Definition Language):
   Used to define database structure

- CREATE
- ALTER
- DROP

Example:
CREATE TABLE students (id INT, name VARCHAR(50));

2. **DML (Data Manipulation Language):**
   Used to modify data

- INSERT
- UPDATE
- DELETE

Example: INSERT INTO students VALUES (1, 'John', 20);

3. **DQL (Data Query Language):**
   Used to retrieve data

- SELECT

Example: SELECT * FROM students;

4. **DCL (Data Control Language):**
   Used for permissions

- GRANT
- REVOKE

Importance of SQL:

- Easy data retrieval
- Supports data manipulation
- Widely used in all database systems